CSP-S 2023 (Senior)

2023-10-21, 14:30–18:30 (4 hours)

Translated from the official Chinese problem booklet (2023 CCF CSP-J/S second round, senior level). All four problems: time limit 1 s, memory limit 512 MB, 100 points each. Input is read from the file `<name>.in` and output is written to `<name>.out`, where `<name>` is the problem code shown in each heading.

Problem 1: Combination Lock (密码锁, lock)

Time limit: 1 s. Memory limit: 512 MB.

Little Y has a combination lock with five dials. As shown in the figure, each dial carries the digits from 0 to 9. Every dial is cyclic from 0 to 9, that is, turning 9 by one position gives 0 or 8.

[Figure 1: a combination lock (photograph omitted).]

Because the campus is fairly safe, Little Y locks the bike as follows: starting from the correct combination, the lock is turned randomly exactly once; each time, either a single dial is turned by some amount, or two adjacent dials are turned simultaneously.

When Little Y chooses to turn two adjacent dials simultaneously, the two dials are turned by the same amount. That is, Little Y can turn the lock from `0 0 1 1 5` into `1 1 1 1 5`, but not into `1 2 1 1 5`.

As time went by, Little Y started to worry about the safety of this way of locking, so Little Y recorded n states of the lock after locking the bike. Note that none of these n states is the correct combination.

To check the safety of this way of locking, Little Y wants to know how many possible correct combinations there are such that each of them can produce all n recorded states of the lock by the locking method described above.

Input

Read the input from the file `lock.in`.

The first line contains a positive integer n, the number of states of the lock after locking.

Each of the next n lines contains five integers describing one state of the lock.

Output

Write the output to the file `lock.out`.

Output one line containing one integer: the number of correct combinations that correspond to these n states of the lock under the given locking method.

Sample 1 input

```
1
0 0 1 1 5
```

Sample 1 output

```
81
```

Explanation of sample 1

There are 81 possible combinations in total. Among them, 45 are obtained by turning one dial and 36 are obtained by turning two dials.

Sample 2

See the files `lock/lock2.in` and `lock/lock2.ans` in the contestant's directory.

Constraints

For all test data: $1 \leq n \leq 8$.

Test cases	$n \leq$	Special property
$1 \sim 3$	1	none
$4 \sim 5$	2	none
$6 \sim 8$	8	A
$9 \sim 10$	8	none

Special property A: it is guaranteed that every correct combination can produce the n given states by turning only one dial.

Problem 2: Match-Three (消消乐, game)

Time limit: 1 s. Memory limit: 512 MB.

Little L is playing a low-budget version of a match-three game. This version of the game is one-dimensional, and each move can only eliminate two adjacent elements.

Now he has a string of length n consisting only of lowercase letters. We call a string erasable if and only if it can be turned into the empty string by performing some number of operations on it.

In each operation, two adjacent equal characters are deleted from the string, and the remaining parts of the string are concatenated.

Little L wants to know how many of the non-empty contiguous substrings of this string are erasable.

Input

Read the input from the file `game.in`.

The first line contains a positive integer n, the length of the string.

The second line contains a string of length n consisting only of lowercase letters, the string asked about in the problem.

Output

Write the output to the file `game.out`.

Output one line containing one integer, the answer to the question.

Sample 1 input

```
8
accabccb
```

Sample 1 output

```
5
```

Explanation of sample 1

There are 5 erasable contiguous substrings in total, namely `cc`, `acca`, `cc`, `bccb` and `accabccb`.

Sample 2

See the files `game/game2.in` and `game/game2.ans` in the contestant's directory.

Sample 3

See the files `game/game3.in` and `game/game3.ans` in the contestant's directory.

Sample 4

See the files `game/game4.in` and `game/game4.ans` in the contestant's directory.

Constraints

For all test data: $1 \le n \le 2 \times 10^6$, and the string asked about consists only of lowercase letters.

Test cases	$n \le$	Special property
$1 \sim 5$	10	none
$6 \sim 7$	800	none
$8 \sim 10$	8000	none
$11 \sim 12$	2×10^5	A
$13 \sim 14$	2×10^5	B
$15 \sim 17$	2×10^5	none
$18 \sim 20$	2×10^6	none

Special property A: every character of the string is chosen independently and uniformly at random from the alphabet.

Special property B: the string consists only of `a` and `b`.

Problem 3: Structs (结构体, struct)

Time limit: 1 s. Memory limit: 512 MB.

Background

In high-level languages such as C++, besides basic types such as `int` and `float`, one can usually also define struct types. In this problem you need to simulate the way structs are defined in a high-level language similar to C++, and compute the corresponding memory usage and related information.

Description

In this language there are 4 basic types: `byte`, `short`, `int` and `long`, occupying 1, 2, 4 and 8 bytes of space respectively.

When defining a struct type, one gives the type name and the members, where for each member one gives, in order, its type and its name. The type may be a basic type or a previously defined struct type. Note that defining a struct type does not define a concrete element, i.e. it does not occupy memory.

When defining an element, one gives the type and the name of the element. An element occupies memory according to the following rules:

- All members of an element are laid out in memory in the order in which they were given in the definition; the same holds for members whose type is a struct.
- To guarantee efficient memory access, the addresses occupied by an element must satisfy the alignment rule: the size of any type, and the starting address of any element of that type in memory, must both be aligned to an integer multiple of the alignment requirement of that type. Concretely:
  - For a basic type, the alignment requirement equals the size of the space it occupies; for example, the type `int` must be aligned to 4 bytes, and similarly for the others.
  - For a struct type, the alignment requirement equals the maximum of the alignment requirements of its members; for example, a struct type containing an `int` and a `short` must be aligned to 4 bytes.

Here is an example (written in C++ syntax):

```
struct d {
    short a;
    int b;
    short c;
};
d e;
```

This code defines the struct type `d` and the element `e`. The element `e` contains three members `e.a`, `e.b` and `e.c`, which occupy the addresses of bytes $0 \sim 1$, $4 \sim 7$ and $8 \sim 9$ respectively. Since the type `d` must be aligned to 4 bytes, `e` occupies the addresses of bytes $0 \sim 11$ and has size 12 bytes.

You need to process n operations, each of which is one of the following four kinds:

1. Define a struct type. Concretely, a positive integer k and strings $s, t_1, n_1, \ldots, t_k, n_k$ are given, where k is the number of members of the type, s is the type name, $t_1, t_2, \ldots, t_k$ are the types of the members in order, and $n_1, n_2, \ldots, n_k$ are the names of the members in order. You need to output the size and the alignment requirement of this struct type, separated by one space.

2. Define an element. Concretely, strings t and n are given, the type and the name of the element respectively. All defined elements are laid out one after another in order, starting from memory address 0, and must satisfy the address alignment rule. You need to output the starting address of the newly defined element.

3. Access an element. Concretely, a string s is given, denoting the element being accessed. As in C++ and similar languages, `.` is used to access a member of a struct type. For example, `a.b.c` means that `a` is a defined element of some struct type having a member named `b`, which is itself of some struct type having a member named `c`. You need to output the starting address of the innermost element accessed in this way.

4. Access a memory address. Concretely, a non-negative integer *addr* is given, the address being accessed. You need to determine whether some element of a basic type occupies this address. If so, output this element in the element-access format of operation 3; otherwise output `ERR`.

Input

Read the input from the file `struct.in`.

Line 1: a positive integer n, the number of operations.

The following lines describe the operations in order; the first positive integer *op* on each line denotes the type of the operation:

- If $op = 1$, first a string s and a positive integer k are given, the type name and the number of members; then each of the next k lines contains two strings t_i and n_i, the type and the name of each member in order.
- If $op = 2$, two strings t and n are given, the type and the name of the element.
- If $op = 3$, a string s is given, the element being accessed.
- If $op = 4$, a non-negative integer *addr* is given, the address being accessed.

Output

Write the output to the file `struct.out`.

Output n lines, the results of the operations in order; the required output is as described in the problem description.

Sample 1 input

```
5
1 a 2
short aa
int ab
1 b 2
a ba
long bb
2 b x
3 x.ba.ab
4 10
```

Sample 1 output

```
8 4
16 8
0
4
x.bb
```

Explanation of sample 1

In the struct type `a`, the member `aa` of type `int` occupies the addresses of bytes $0 \sim 3$, and the member `ab` of type `short` occupies the addresses of bytes $4 \sim 5$. Since its alignment requirement is 4 bytes, its size is 8 bytes. In the same way one computes that the struct type `b` has size 16 bytes and alignment requirement 8 bytes.

(Translator's note: the sample explanation above is translated verbatim from the original booklet, which lists the members of `a` as `aa` of type `int` and `ab` of type `short`, whereas the sample input declares `short aa` and `int ab`.)

Sample 2

See the files `struct/struct2.in` and `struct/struct2.ans` in the contestant's directory.

Explanation of sample 2

In the second operation of type 4, the accessed memory address lies exactly in a "hole" left for address alignment, so no element of a basic type occupies it.

Sample 3

See the files `struct/struct3.in` and `struct/struct3.ans` in the contestant's directory.

Constraints

For all data, $1 \leq n \leq 100$, $1 \leq k \leq 100$ and $0 \leq addr \leq 10^{18}$.

All struct type names, member names and element names consist of at most 10 lowercase letters, and none of them is `byte`, `short`, `int` or `long` (i.e. they do not coincide with the names of basic types).

All struct type names and element names are pairwise distinct, and the member names within one struct are pairwise distinct. However, different structs may have members with the same name, and a member name inside a struct may coincide with the name of a struct type or of an element.

It is guaranteed that all operations conform to the rules and requirements stated in the problem; i.e. a struct definition never uses a non-existent type, no non-existent element or member is accessed, and so on.

It is guaranteed that the size of every struct, and the highest memory address occupied by any defined element, do not exceed 10^{18}.

Test case number	Special properties
1	A, D
$2 \sim 3$	A
$4 \sim 5$	B, D
$6 \sim 8$	B
$9 \sim 10$	C, D
$11 \sim 13$	C
$14 \sim 16$	D
$17 \sim 20$	none

Special property A: there is no operation of type 1.
Special property B: there is exactly one operation of type 1.
Special property C: all member types given in operations of type 1 are basic types.
Special property D: the only basic type used is `long`.

Hint

The alignment requirement and the size of a struct type are formally defined as follows.

- Suppose the struct has k members with sizes $s_1, \ldots, s_k$ and alignment requirements $a_1, \ldots, a_k$.
- Then the alignment requirement of the struct is $a = \max\{a_1, \ldots, a_k\}$.
- Further suppose the address offsets of these members in the layout are $o_1, \ldots, o_k$. Then:
  - $o_1 = 0$;
  - for $i = 2, \ldots, k$, o_i is the smallest value such that $o_{i-1} + s_{i-1} \leq o_i$ and a_i divides o_i;
  - the size s of the struct is the smallest value such that $o_k + s_k \leq s$ and a divides s.

The memory layout of defined elements is formally defined as follows.

- Suppose the i-th defined element has size s_i, alignment requirement a_i and starting address b_i.
- Then $b_1 = 0$, and for $2 \leq i$, b_i is the smallest value such that $b_{i-1} + s_{i-1} \leq b_i$ and a_i divides b_i.

Problem 4: Planting Trees (种树, tree)

Time limit: 1 s. Memory limit: 512 MB.

You are a forest ranger. One day you receive a task: plant trees on the plots of a forest and tend them until the trees grow to the specified heights.

The map of the forest has n plots, and plot number 1 is connected to the entrance of the forest. There are $n - 1$ roads connecting these plots, so that every plot can be reached from every other plot via the roads. Initially there are no trees on any plot.

Your goal is to plant one tree on every plot, and to make the tree on plot i grow to a height of at least a_i metres.

Each day you may choose a plot that has no tree yet and is directly adjacent to some plot that already has a tree (i.e. connected to it by a single road), and plant a tree of height 0 metres there.

If every plot has already been planted, you do nothing on that day. In particular, on day 1 you can only plant on the empty plot number 1.

For each plot, starting from the day on which its tree is planted, the tree on that plot grows by a certain height every day. Because of differing climate and soil conditions, on day x the tree on plot i grows by $\max(b_i + x * c_i, 1)$ metres. Note that x is counted from the first day of the whole task, not from the first day after this tree was planted.

You want to know: what is the minimum number of days needed to complete your task?

Input

Read the input from the file `tree.in`.

The first line contains a positive integer n, the number of plots in the forest.

Each of the next n lines contains three integers a_i, b_i, c_i describing one plot, with the meaning given in the problem description.

Each of the next $n - 1$ lines contains two positive integers u_i, v_i denoting a road connecting plots u_i and v_i.

Output

Write the output to the file `tree.out`.

Output one line containing one positive integer, the minimum number of days needed to complete the task.

Sample 1 input

```
4
12 1 1
2 4 -1
10 3 0
7 10 -2
1 2
1 3
3 4
```

Sample 1 output

```
5
```

Explanation of sample 1

Day 1: plant a tree on plot 1; the tree on plot 1 grows to 2 metres.

Day 2: plant a tree on plot 3; the trees on plots 1, 3 grow to 5, 3 metres respectively.

Day 3: plant a tree on plot 4; the trees on plots 1, 3, 4 grow to 9, 6, 4 metres respectively.

Day 4: plant a tree on plot 2; the trees on plots 1, 2, 3, 4 grow to 14, 1, 9, 6 metres respectively.

Day 5: the trees on plots 1, 2, 3, 4 grow to 20, 2, 12, 7 metres respectively.

Sample 2

See the files `tree/tree2.in` and `tree/tree2.ans` in the contestant's directory.

Sample 3

See the files `tree/tree3.in` and `tree/tree3.ans` in the contestant's directory.

Sample 4

See the files `tree/tree4.in` and `tree/tree4.ans` in the contestant's directory.

Constraints

For all test data: $1 \le n \le 10^5$, $1 \le a_i \le 10^{18}$, $1 \le b_i \le 10^9$, $0 \le |c_i| \le 10^9$ and $1 \le u_i, v_i \le n$. It is guaranteed that there is a way to complete the task within 10^9 days.

(In the original table the entries 20, 10^5 and the property A of test cases $5 \sim 8$ are merged cells spanning several rows; they are repeated on each row below.)

Test case number	$n \le$	Special property
1	20	A
$2 \sim 4$	20	none
$5 \sim 6$	500	A
$7 \sim 8$	10^5	A
$9 \sim 10$	10^5	B
$11 \sim 13$	10^5	C
$14 \sim 16$	10^5	D
$17 \sim 20$	10^5	none

Special property A: for all $1 \le i \le n$, $c_i = 0$.
Special property B: for all $1 \le i < n$, $u_i = i$ and $v_i = i + 1$.
Special property C: no plot has more than 2 roads directly connected to it.
Special property D: for all $1 \le i < n$, $u_i = 1$.

Source

The problems were translated from the official statement PDF (`download.php?type=statement&id=...`) displayed on the following QOJ problem pages:

- Problem 1 (lock): `https://qoj.ac/problem/7813`
- Problem 2 (game): `https://qoj.ac/problem/7814`
- Problem 3 (struct): `https://qoj.ac/problem/7815`
- Problem 4 (tree): `https://qoj.ac/problem/7816`

Contest page: `https://qoj.ac/contest/1428` (CSP-S 2023).